Rezolake — architecture

Rezolake is a master-data-management (entity resolution) app that runs entirely inside one Databricks workspace: an **AppKit app** for configuration and stewardship, a **serverless engine job** that resolves records into mastered entities, **Delta tables in Unity Catalog** as the system of record, and **Lakebase Postgres** for the app's operational state.

Contents

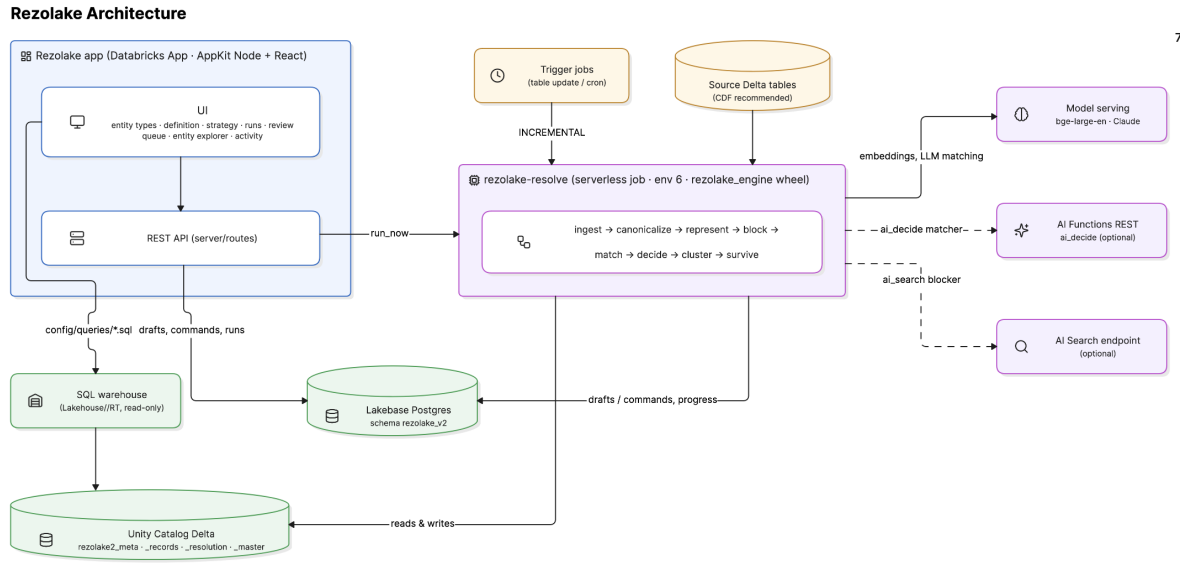
1. Principles
2. Runtime components
3. Data architecture (incl. provenance and history)
4. Resolution pipeline
5. Strategies and pluggable components
6. Stewardship
7. The app
8. Operations
9. Open items

1. Principles

- **Delta is the MDM system of record.** Source history, canonical records, candidate pairs, scores, decisions, human constraints, membership, golden attributes with provenance and run manifests are all Delta tables. History is appended; "current" tables are derived and can be rebuilt.
- **Lakebase holds operational state only:** drafts, steward commands, run requests and progress, review notes, orchestration policies. Losing it loses no mastered data.
- **Every write to mastered data goes through the engine job.** The app reads Delta through a SQL warehouse (Lakehouse//RT in prod, which is read-only) and writes only to Lakebase. Steward actions are queued as commands that the next engine run applies.
- **Human decisions are durable.** Steward actions become constraints and overrides that every later automated run must respect, so reruns never undo a person's decision.

- **Everything a run used is versioned.** Entity definitions and resolution strategies are published as immutable versions; each run records the versions and components it used in its manifest.
- **Stages are pluggable.** Blocking, matching, decision, clustering and survivorship are components resolved by name from the published strategy, so any one can be replaced without touching the others.

2. Runtime components



Compon - ent	What it is	Notes
App	AppKit app (`server/` , `client/`), deployed by the bundle as `rezolake`	Runs as its service principal; source previews run as the signed-in user (`*.obo.sql`)
Engine job	`rezolake-resolve` , one `python_wheel_task` on serverless **environment 6**	Max one concurrent run, queued. Modes in §4
Trigger jobs	One per entity type (`rezolake-trigger-<id>`), created by the app when orchestration is enabled	A "doorbell": its only task runs the engine job in `INCREMENTAL` mode on a table update or cron
SQL warehouse	Lakehouse//RT in prod (`rezolake-rt-wh`); any serverless/pro warehouse works	Read-only for the app. RT can't read default-storage/external tables
Lakebase	Autoscaling project (`projects/rezolake-postgres-db` in prod), branch `production`	App creates and owns schema `rezolake_v2` ; grants the engine identity DML
Model serving	`databricks-bge-large-en` (embeddings), `databricks-claude-sonnet-4-6` (LLM matcher)	Per-strategy, changeable in the Strategy tab

Compon - ent	What it is	Notes
AI Functions	`POST /api/2.0/ai-functions/ai-decide` (Beta; workspace preview)	Only for the `ai_decide_matcher`
AI Search	Vector Search endpoint `rezolake-vs-endpoint`	Only for the `ai_search_blocker` ; bootstrap creates it with `--with-ai-search`

3. Data architecture

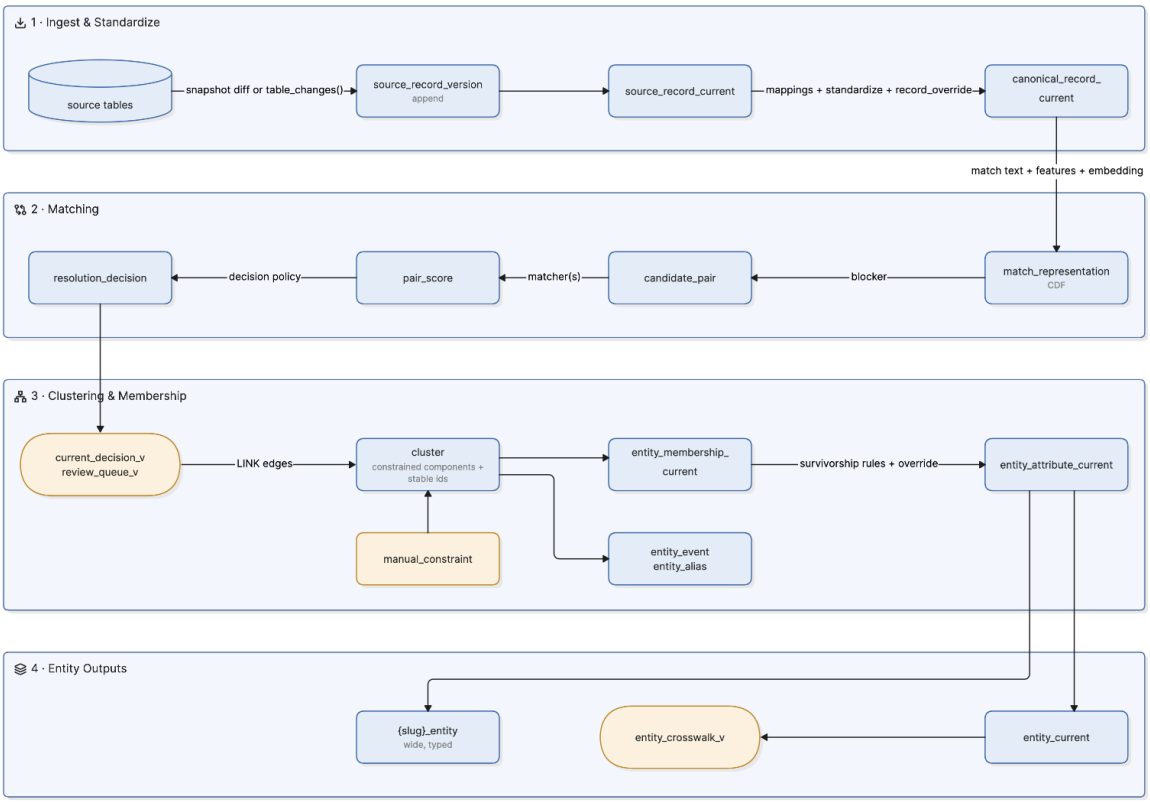
3.1 Schemas

All Delta objects live in one catalog (the bundle's `catalog` variable), in four schemas created by the engine's `SETUP` mode (`engine/src/rezolake_engine/ddl.py`):

Schema	Holds
`rezolake2_meta`	Published definitions and strategies, source checkpoints, run manifests
`rezolake2_records`	Source ledger, canonical records, record-level overrides, match representation (embeddings)
`rezolake2_resolution`	Candidate pairs, pair scores, decisions, human constraints, match state, membership, entity events and aliases
`rezolake2_master`	Mastered entities, golden attributes with provenance, survivorship overrides, per-type wide tables

The app's service principal gets `USE CATALOG` and `USE SCHEMA, SELECT` on all four; it never writes them.

3.2 Data flow



3.3 Tables

`rezolake2_meta`

Table	Grain	Written by	Purpose
`entity_definition_version`	entity type × version	`PUBLISH`	Frozen definition JSON: attributes (id, data/semantic type, matching, display, survivorship, standardize) and sources (table, primary key, column → attribute mappings, priority)
`strategy_version`	entity type × version	`PUBLISH`	Frozen strategy JSON: embedding endpoint + blocker, matchers, decision, clusterer, survivorship components with versions and config
`source_checkpoint`	entity type × source	ingest (committed at run end)	Last source Delta version read and the read mode (`snapshot` / `cdf`)
`resolution_run`	run	every run	Manifest (definition/strategy versions, component refs, mode, sources), status, metrics
`resolution_run_source`	run × source	ingest	Version range read and inserted / updated / deleted counts

`rezolake2_records`

Table	Grain	Purpose
`source_record_version`	record × observed change	Append-only ledger of raw payloads (JSON), with source commit version and change type
`source_record_current`	record	Latest payload per record, `is_deleted` when it vanished from the source
`canonical_record_current`	record	Canonical `attributes` map keyed by **attribute_id**, unmapped columns in `additional_attributes`, `record_hash` of the canonical form
`record_override`	override	Steward edits to one record's attribute value; applied last in canonicalization so they survive re-ingest
`match_representation`	record	`match_text`, normalized `features` map, `embedding ARRAY<FLOAT>` (L2-normalized), `canonical_hash`, `embedding_model`. CDF on (the AI Search index syncs from it)

`rezolake2_resolution`

Table / view	Grain	Purpose
`candidate_pair`	run × pair	Blocker output: ordered pair (left < right), query record, rank, `block_score`, blocker id/version, both records' hashes
`pair_score`	run × pair × matcher	`score` = P(match) in [0, 1], `evidence` JSON (rationale for the LLM matcher), `trace_id`, matcher id/version, hashes. Negatives are kept
`resolution_decision`	run × pair	LINK / REVIEW / NO_LINK from the decision policy, with score and reason, and its provenance: `strategy_version`, `blocker` ref and `scores` (JSON: each matcher ref, score and the run that produced it — reused scores keep their original run)
`current_decision_v`	pair	Latest decision per pair with its provenance and record hashes; `is_current` only while both records still have the hashes it was decided on and neither is deleted
`review_queue_v`	pair	Current REVIEW decisions with no human constraint on the pair
`manual_constraint / active_constraint_v`	constraint	MUST_LINK, CANNOT_LINK and ISOLATE rules from stewards (active until revoked)
`record_match_state`	record	

Table / view	Grain	Purpose
		`blocked_hash` : the canonical hash at which the record was last resolved (drives the incremental work set)
`entity_membership_current`	record	Record → entity id
`entity_event`	event	CREATE / MERGE / SPLIT / RETIRE with source and target entity ids, run and command ids (lineage)
`entity_alias`	old id	Retired entity id → surviving id, so old ids keep resolving after merges

`rezolake2_master`

Table / view	Grain	Purpose
`entity_current`	entity	Anchor record, member and source counts, `display_name` (first two display attributes), `golden` JSON
`entity_attribute_current`	entity × attribute	Golden value with provenance: winning record and source, rule, manual-override flag, all observed values
`survivorship_override`	override	Steward overrides of a golden value and pinned golden records
`<slug>_entity`	entity	Consumer-facing wide table with typed columns per attribute (rebuilt by survive)
`entity_crosswalk_v`	record	Source id + source key → entity id, with the anchor flag

3.4 Identifiers and change detection

Value	Definition	Used for
`record_id`	`sha256(entity_type_id | source_id | normalized primary key)`	Stable across re-ingest; the key of every record table
`pair_id`	`sha256(min(a,b) | max(a,b))`	Order-free pair key
`record_hash`	hash of the canonical record	A record only re-embeds, re-blocks and re-matches when this changes
`blocked_hash`	`record_hash` when last resolved	Work set (every mode) = live records where `blocked_hash ≠ record_hash` , plus `REMATCH` records, or every live record with `rematch_all` . Cleared when a record's scoring fails, so the next run retries it
entity id	stable UUID	

Value	Definition	Used for
		Survives membership changes: on merge the older entity id survives (the other becomes an alias); on split the side holding the anchor keeps the id

Scores are reused: a pair already scored by the same matcher id and version at the same pair of record hashes is not scored again, so reruns only pay for what changed.

3.5 Provenance and history

- **Nothing that decided a match is overwritten.** Candidates, scores and decisions are append-only; each carries the component id and version that produced it (prompt version included for the LLM matcher).
- **Each decision names what made it** — strategy version, blocker and the exact scores — and the UI shows the evidence of those scores (matched on pair, record versions and matcher), labelled "strategy vN · matcher".
- **Settled matches stay settled.** A new strategy (matcher, blocker, decision thresholds or embedding model) applies only to records that change or arrive after it is published. `FULL` re-reads sources, re-embeds and rebuilds entities but does not re-decide settled pairs. Re-deciding everything with the current strategy is an explicit choice: `rematch_all` (Runs tab switch), recorded in the run manifest; the old decisions stay in `resolution_decision` as history.
- Entity membership and golden records are always rebuilt from the current decisions and constraints, so they change only when decisions, constraints, overrides or the clusterer / survivorship configuration change.

3.6 Lakebase (schema `rezolake_v2`, owned by the app)

Table	Purpose
`entity_type`	Entity types (id, name, slug, kind, status, published versions)
`entity_definition_draft` / `strategy_draft`	Editable drafts; `PUBLISH` freezes them into Delta
`run_request`	Every run the app or a trigger started: mode, params, job run id, status, stage, progress, metrics
`resolution_command`	Queued steward commands (type, payload, status, run that applied it, error)
`review_note`	Steward notes, flags and "unsure" verdicts on review pairs
`orchestration_policy`	Per-type automation: trigger mode (table update / cron), debounce and minimum interval, trigger job id

4. Resolution pipeline

4.1 Modes (job parameter `mode`)

Mode	Does
`SETUP`	Creates schemas, tables and views; grants the app service principal; creates the trace experiment
`PUBLISH`	Freezes the Lakebase drafts into new `entity_definition_version` / `strategy_version` rows (no-op if unchanged)
`INCREMENTAL`	Applies pending commands, ingests source changes, resolves only the work set
`FULL`	Same, but snapshots every source and re-embeds every record; still re-matches only the work set
`LIMITED`	`INCREMENTAL` with a cap on the work set (`max_records`); also accepted with `FULL`
`COMMAND`	Applies commands and re-resolves what they touched, without ingesting

`rematch_all=true` (any resolve mode) puts every live record in the work set, re-deciding settled pairs with the current strategy (scores at unchanged record versions are still reused). Every resolve run first applies any pending table migrations (`ddl.migrate`).

4.2 Stages

Stage	Input → output	Notes
commands	`resolution_command` → constraints / overrides	See §6
ingest	source tables → source ledger	First run, `FULL` or no CDF: snapshot diff (deletes detected by absence). Otherwise `table_changes(last+1..current)`. Checkpoints advance only when the whole run succeeds
canonicalize	payload → canonical record	Column mappings by attribute id, per-attribute casing (`standardize`), extensions, then record overrides
represent	canonical record → match representation	Match text from matching attributes, normalized features per semantic type, embedding via `ai_query(embedding_endpoint)` for changed records only
block	work set → `candidate_pair`	Strategy blocker; `prepare()` first (e.g. sync an index), then candidates
match	candidate pairs → `pair_score`	Every strategy matcher, with score reuse. Scored in batches of 200 query records, each written as it completes, so a cancelled or failed run keeps its work.

Stage	Input → output	Notes
		Between batches a run yields to queued steward work (below)
decide	scores → `resolution_decision`	Threshold policy: LINK ≥ link threshold (0.85), REVIEW ≥ review threshold (0.5), else NO_LINK; scores of several matchers combined by max / min / mean
cluster	LINK edges + constraints → membership	Constrained connected components (CANNOT_LINK > MUST_LINK > automatic LINK by descending score), stable ids, events and aliases. With `no_link_veto_below`, a NO_LINK scored below it acts as a soft CANNOT_LINK for automatic links, so chains of links can't join confidently different records (a human MUST_LINK still can)
survive	members + rules + overrides → golden record	Per-attribute rules (source priority, most recent non-null, most common, longest), value overrides, pinned records; rebuilds `<slug>_entity`

5. Strategies and pluggable components

A strategy is a JSON document (`representation`, `blocker`, `matchers[]`, `decision`, `clusterer`, `survivorship`), each a `{component, version, config}` reference. The engine resolves names through `components/registry.py`; contracts are in `components/base.py`. The default is in `DEFAULT_STRATEGY` (`definition.py`); the Strategy tab edits a draft that `PUBLISH` freezes.

Stage	Components	Default
Representation	`embedding_endpoint`	`databricks-bge-large-en`
Blocker	`nearest_by_blocker` — one SQL `NEAREST BY` join over `match_representation`, cosine similarity, no index (k + 1 then drop self; `EXACT` or `APPROX`) · `ai_search_blocker` — AI Search ANN over its own Delta Sync index, which its `prepare()` creates and syncs	`nearest_by_blocker` (10 candidates, similarity ≥ 0.5)
Matcher	`llm_entity_matcher` — DSPy prompt with entity-kind rules, one call per record with up to 10 candidates, rationale per decision, MLflow traces · `ai_decide_matcher` — `ai_decide` decision model per pair via REST, P(match) only, no rationale (Beta)	`llm_entity_matcher` (Claude Sonnet 4.6, 16 concurrent)
Decision	`threshold_policy`	link 0.85 / review 0.5 / combine max
Clusterer	`constrained_components` (v2 adds the `no_link_veto_below` guard)	v2 with `no_link_veto_below` 0.2 for new strategies; existing

Stage	Components	Default
		strategies keep v1 behaviour until changed
Survivorship	`attribute_rules`	—

Blocker scores differ by component: `nearest_by_blocker` reports cosine similarity; AI Search reports `1 / (1 + L2²)` = `1 / (3 - 2·cos)` for normalized vectors. They agree only at 0.5.

Adding a component (for example an XGBoost matcher over string distances and cosine similarity):

1. Implement the contract from `base.py` — for a matcher, `score_groups(ctx, groups, texts) → list[ScoredPair]`, one `ScoredPair` per candidate with `score` = P(match); put `{"error": ...}` in `evidence` for pairs that could not be scored (they are retried next run).
2. Register it in `registry.py` and give it a `component_id` and `version`; bump the version whenever its behaviour changes so old scores are not reused.
3. Offer it in the Strategy tab (`client/src/pages/StrategyTab.tsx`).

Matchers currently receive each record's `match_text`. A feature-based model will also want the normalized `features` map and the embedding from `match_representation`; extend the matcher input when the first such matcher is built.

Matcher trade-offs (measured on prod data, 2026-10)

	LLM matcher	ai_decide matcher
Agreement	baseline	471–472 / 474 pairs agree on `customer` once its prompt includes the identifier-conflict rule
Batch speed in the job	~13 pairs/s; ~2.2 pairs/s sustained on a 31k-pair run when Claude's per-minute token limit bites (39% of calls retried)	~2.5 pairs/s sustained from serverless jobs (p99 ≈ 50 s), though ~54 pairs/s from outside the workspace and ~100/s in short bursts
Explanations	rationale per pair, MLflow trace	none
Products (Amazon–Google benchmark, 1,300 true pairs; blocking recall ceiling 0.92)	pair-level F1 0.71 at 0.85 (0.73 at 0.5–0.7); entity-level F1 0.71 (P 0.67 / R 0.76) with the clusterer guard at 0.2; 0.73 (P 0.65 / R 0.82) with the link threshold lowered to 0.7	pair-level F1 0.47 at 0.85 on the same pairs (best 0.57 at 0.5); entity-level F1 0.38 without the guard

6. Stewardship

UI actions call `POST /api/entity-types/:id/commands`, which queues a `resolution_command` in Lakebase and starts a `COMMAND` run (or joins a run that is still pending, so bursts of actions share one run). The engine applies commands in order and translates each into durable Delta rows:

Command	Writes	Effect
`ACCEPT_MATCH` / `REJECT_MATCH`	`manual_constraint` MUST_LINK / CANNOT_LINK	Overrides the automatic decision for that pair
`MERGE_ENTITIES`	MUST_LINK from the surviving anchor to every record of the merged entities; revokes CANNOT_LINKs between the groups and ISOLATEs on the anchors	Entities join and stay joined on human links alone; older id survives, other becomes an alias
`SPLIT_ENTITY`	CANNOT_LINK between every moved and remaining record; MUST_LINK among the moved records	Moved records form a new entity and stay together
`ISOLATE_RECORD`	ISOLATE	Record never joins another entity automatically
`REVOKE_CONSTRAINT`	deactivates a constraint	Automatic decisions apply again
`OVERRIDE_ATTRIBUTE`	`survivorship_override` (value)	Golden value fixed regardless of rules
`PIN_GOLDEN_RECORD`	`survivorship_override` (record)	Golden values taken from that record
`EDIT_RECORD`	`record_override`	One record's value corrected at canonicalization; record re-matched
`REMATCH`	—	Forces the records (or an entity's records) back into the work set

7. The app

- **REST API** (`server/routes/api.ts` , `orchestration.ts`): entity types, drafts, publish, runs, commands, review notes, Unity Catalog browsing, orchestration policies. Lakebase only, plus starting the engine job.
- **Reads** through `config/queries/*.sql` (AppKit analytics, typed in `shared/appkit-types/`): `entity_type_stats` , `entities_page` (search / filter / sort / paginate), `entity_size_distribution` , `entity_detail` , `entity_members` ,

`entity_attributes`, `entity_edges`, `entity_events`, `crosswalk_lookup`, `review_queue`, `run_history`, `source_preview.obo` (as the user).

- **Entity explorer**: summary strip, server-side filtered list, the selected entity in a resizable side panel (full page on narrow screens), with tabs for *Compare records* (golden vs every record, differences highlighted, edits and overrides), *Why together?* (graph of decisions and constraints; click a record for its details; split suggestions from a maximum-spanning-tree cut), raw payloads and lineage. Merge searches for the other entity and merges the entities or links two chosen records.

8. Operations

- **Deploy:** `databricks bundle deploy -t <target>` then `databricks bundle run app -t <target>`; new environments via `scripts/setup.py` (which runs `scripts/bootstrap.py` and `scripts/smoke_test.py`) (see SETUP.md).
- **Engine version:** bump `engine/pyproject.toml` on every engine change so serverless reinstalls the wheel.
- **One run at a time:** the engine job has `max_concurrent_runs: 1` with queueing, shared by every entity type, so steward commands wait for the run in front of them. To keep that short, a non-`COMMAND` run checks between scoring batches whether a steward command (any entity type) is waiting on a queued run; if so it stops matching, finishes the run with what it scored (unscored records keep a cleared `blocked_hash`), and queues an `INCREMENTAL` **continuation** run (`trigger_source = continuation`) behind the steward run. The Runs tab shows the whole queue across entity types (`GET /api/runs/active`).
- **Serverless environment 6** (Python 3.12, Databricks Connect 19.1, `mlflow-skinny` preinstalled). Full `mlflow` failed to install on environment 4 in October 2026, which is why the engine depends on `mlflow-skinny` only. To debug an install failure, submit one task per dependency (at most 10 environments per submitted run). Environment 6 disables Py4J/`dbutils` APIs; the engine uses none.
- **Rate limits:** Foundation Model APIs enforce a workspace output-tokens-per-minute limit; the LLM matcher backs off 10–60 s on `REQUEST_LIMIT_EXCEEDED`. `ai_decide` returns 429 above ~100 calls/s.
- **AI Decide** needs the workspace preview enabled (bootstrap warns if it is off).
- **Scale:** `NEAREST BY EXACT` on a Small warehouse: 100k × 1k in 9 s, 100k × 10k in 67 s, 1M × 1k in 47 s. Incremental runs are cheap; a `FULL` re-match of millions against millions needs a larger warehouse or the AI Search blocker. `FULL` also re-embeds every record.
- **Query types:** `shared/appkit-types/analytics.d.ts` is generated from each query's `-- @param` sample values, which name the prod catalog; regenerate against prod after editing `config/queries/*.sql` (see SETUP.md).

9. Open items

Item	Notes
Over-merging through transitive links	The `no_link_veto_below` guard (clusterer v2) addresses it for strategies that enable it; the split suggestion handles existing entities by hand
`ai_decide` throughput from jobs	Service-side; to be reported to the AI Functions team. Until fixed it stays a matcher option, not the default
Real-time match / search-before-create	Deferred. Measured design: candidates via `NEAREST BY` on the warehouse (~2 s with embedding) + parallel matcher calls
Feature-based matchers	Deferred. Extend the matcher input with features and embeddings (§5)
Rationale for `ai_decide` reviews	Deferred. Optionally ask the LLM only for pairs in the review band